

LAPORAN TUGAS 4
SISTEM OPERASI



Dosen pengampu :
Triawan Adi Cahyono M.Kom

Disusun Oleh:
Nama : Putri Suci Rahmadhani
Nim : 2410651008
Tanggal : 31 Oktober 2025

PRODI TEKNIK INFORMATIKA FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

1.1. TUJUAN

1. Mahasiswa dapat melihat perbedaan hasil waktu tunggu (waiting time) dan waktu penyelesaian (turnaround time) dari setiap algoritma.
2. Mempelajari kelebihan dan kekurangan tiap metode sesuai kondisi proses yang berbeda-beda.
3. Mengetahui bagaimana pemilihan algoritma yang tepat dapat memengaruhi kinerja sistem.
4. Melatih kemampuan untuk menganalisis performa, keadilan (fairness), dan efisiensi penjadwalan CPU.
5. Membiasakan penggunaan program C di Ubuntu untuk menjalankan simulasi nyata dan menyimpan hasil percobaan sebagai bukti analisis.

1.2.DASAR TEORI

1. Fcfs (first come first served)

Adalah algoritma bekerja seperti antrian supermarket siapa yang datang terlebih dahulu akan dijalankan lebih dulu. Sederhana tapi bisa menyebabkan proses yang datang belakangan menunggu lama jika proses pertama sangat panjang.

2. Sjf (shortest job first)

Algoritma SJF memilih proses dengan waktu eksekusi paling singkat untuk dijalankan lebih dulu. Metode ini lebih efisien karena bisa mengurangi waktu tunggu rata-rata, tetapi kurang adil bagi proses panjang.

3. Round robin

Algoritma Round Robin memberikan waktu eksekusi bergilir untuk setiap proses dengan batas waktu tertentu (time quantum). Jika waktu habis dan proses belum selesai, proses dimasukkan kembali ke antrian. Metode ini lebih adil karena semua proses mendapat giliran.

1.3.TUGAS AKHIR

1. **Tugas 1 Eksperimen dengan data berbeda**

- a. Codingan

diambil dari praktikum 1 yang fcfs

```

#include <stdio.h>
// Struktur untuk menyimpan data proses
struct Process {
    int pid; // ID Proses
    int arrival_time; // Waktu kedatangan
    int burst_time; // Waktu eksekusi
    int waiting_time; // Waktu menunggu
    int turnaround_time; // Waktu total (dari datang sampai selesai)
    int completion_time; // Waktu selesai
};

int main() {
    int n; // Jumlah proses

    printf("=== SIMULASI ALGORITMA FCFS ===\n");
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);

    struct Process proc[n];

    // Input data proses
    printf("\n");
    for(int i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        printf("Proses P%d\n", proc[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &proc[i].arrival_time);
        printf("Burst Time: ");
        scanf("%d", &proc[i].burst_time);
        printf("\n");
    }

    // Hitung completion time, waiting time, dan turnaround time
    int current_time = 0;

    for(int i = 0; i < n; i++) {
        // Jika CPU idle, loncat ke waktu kedatangan proses
        if(current_time < proc[i].arrival_time) {
            current_time = proc[i].arrival_time;
        }
    }
}

```

b. Penjelasan algoritma

- Pada data set 1, proses pertama datang paling awal dan memiliki waktu eksekusi yang paling awal karena menggunakan algoritma FCFS maka p1 langsung dijalankan akibatnya proses p2 dan p3 harus menunggu cukup lama karena mereka datang Ketika p1 masih berjalan hal ini membuat waktu tunggu p2 dan p3 menjadi tinggi. Meskipun FCFS bersifat adil dalam urutan kedatangan, namun hasilnya tidak efisien jika ada proses dengan waktu eksekusi jauh lebih Panjang diawal.
- Pada data set 2, waktu eksekusi tiap proses lebih bervariasi tetapi tidak terlalu jauh berbeda. Karena p1 datang lebih dulu, FCFS tetap mengeksekusinya pertama, setelah itu proses lain dijalankan sesuai urutan kedatangannya. Hasilnya, waktu tunggu setiap proses lebih seimbang dibanding data set 1, karena tidak ada proses yang sangat lama di awal. Namun algoritma SJF bisa membuat hasil lebih efisien, sebab SJF akan memprioritaskan proses dengan waktu eksekusi lebih pendek terlebih dahulu.

c. Hasil output program

- Output data set 1

```

madha_27@DESKTOP-S58N8I3:~$ nano fcfs.c
madha_27@DESKTOP-S58N8I3:~$ gcc fcfs.c -o fcfs
madha_27@DESKTOP-S58N8I3:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0 10 15 23
madha_27@DESKTOP-S58N8I3:~$

```

- Output data set 2

```

madha_27@DESKTOP-S58N8I3:~$ gcc fcfs.c -o fcfs
madha_27@DESKTOP-S58N8I3:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8     16     14      6
P4      3      6     22     19     13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0 5 8 16 22
madha_27@DESKTOP-S58N8I3:~$

```

2. Tugas 2 Analisis eksperimen ketiga algoritma

- a. Algoritma mana yang memberikan average waiting time terkecil?
SJF karena pakai urutan pilih proses paling pendek dari yang ada maksudnya selalu mengeksekusi proses berdurasi paling singkat terlebih dahulu
- b. Mengapa hasil algoritma berbeda-beda?
Karena tiap algoritma memilih urutan eksekusi yang berbeda-beda dan urutan itu langsung mempengaruhi siapa menunggu berapa lama, intinya:
 - FCFS pakai urutan kedatangan (siapa yang datang terlebih dahulu itu yang dikerjakan awal).
 - SJF pakai urutan proses paling pendek (beberapa data divari waktu eksekusinya yang paling pendek itu yang dieksekusi di awal).
 - Round Robin pakai urutan bagi waktu secara bergiliran (lebih adil atau responsive tapi banyak pindah konteks jika quantum kecil).
- c. Kapan sebaiknya menggunakan algoritma FCFS?
Saat system bersifat batch (pekerjaan dijalankan berurutan) dan proses punya durasi yang mirip, atau bisa saat mengutamakan kesederhanaan implementasi dan sedikit overhead (penjadwalan kompleks).
- d. Kapan sebaiknya menggunakan algoritma SJF?
Saat tahu lama eksekusi tiap tugas atau tujuan utama adalah meminimalkan rata-rata waktu tunggu.
- e. Kapan sebaiknya menggunakan algoritma round robin?
Saat system harus responsif dan melayani banyak tugas/permintaan kecil secara adil, quantum dipilih sedang.

3. Tugas 3 Modifikasi program

- a. Codingan
Codingan sama seperti praktikum SJF

```

GNU nano 4.8
#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    bool is_completed;
};

int main() {
    int n;
    printf("=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===\n");
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);

    struct Process proc[n];

    // Input data proses
    printf("\n");
    for(int i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        proc[i].is_completed = false;
        printf("Proses P%d\n", proc[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &proc[i].arrival_time);
        printf("Burst Time: ");
        scanf("%d", &proc[i].burst_time);
        printf("\n");
    }

    int current_time = 0;
    int completed = 0;
    int sequence[n]; // Menyimpan urutan eksekusi

    // Proses penjadwalan SJF
    while(completed < n) {
        int shortest_job = -1;
        int min_burst = 99999;

```

Hanya menambahkan gantt chart, simpan hasil dan input dari file.

```

// Menampilkan hasil ke layar
printf("\nHasil Penjadwalan SJF:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", proc[i].pid, proc[i].arrival_time, proc[i].burst_time,
        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);
}

// Menampilkan Gantt Chart
printf("\nGantt Chart:\n");
printf(" ");
for (int i = 0; i < seq_index; i++) {
    printf(" P%d |", sequence[i]);
}
printf("\n");

int time = 0;
for (int i = 0; i < seq_index; i++) {
    time += proc[sequence[i] - 1].burst_time;
    printf(" %d", time);
}
printf("\n");

float avg_wt = (float)total_waiting / n;
float avg_tat = (float)total_turnaround / n;
printf("\nRata-rata Waiting Time: %.2f", avg_wt);
printf("\nRata-rata Turnaround Time: %.2f", avg_tat);

// Menyimpan hasil ke file
outputFile = fopen("hasil_sjf.txt", "w");
fprintf(outputFile, "=== HASIL PENJADWALAN SJF ===\n");
fprintf(outputFile, "PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    fprintf(outputFile, "P%d\t%d\t%d\t%d\t%d\t%d\n", proc[i].pid, proc[i].arrival_time, proc[i].burst_time,
        proc[i].completion_time, proc[i].turnaround_time, proc[i].waiting_time);
}
fprintf(outputFile, "\nAverage Waiting Time = %.2f\n", avg_wt);
fprintf(outputFile, "Average Turnaround Time = %.2f\n", avg_tat);
fclose(outputFile);

printf("\nHasil juga disimpan ke file hasil_sjf.txt\n");

```

b. Penjelasan algoritma

- Cpu akan mengecek proses mana yang sudah datang dan memiliki burst time terkecil.
- Proses itu akan dieksekusi sampai selesai.
- Setelah selesai cpu melanjutkan ke proses berikutnya dengan burst time terkecil yang sudah datang.
- Setelah semua selesai program menampilkan table hasil dan menyimpan ke file hasil_sjf.txt

c. Hasil output program

```
madha_27@DESKTOP-S58N8I3:~$ nano sjf.c
madha_27@DESKTOP-S58N8I3:~$ gcc sjf.c -o sjf
madha_27@DESKTOP-S58N8I3:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
File input_sjf.txt tidak ditemukan! Masukkan data manual.
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time : 10

Proses P2
Arrival Time: 1
Burst Time : 5

Proses P3
Arrival Time: 2
Burst Time : 8

Hasil Penjadwalan SJF:
PID   AT   BT   CT   TAT   WT
P1     0   10   10   10     0
P2     1    5   15   14     9
P3     2    8   23   21    13

Gantt Chart:
| P1 | P2 | P3 |
0  10  15  23

Rata-rata Waiting Time: 7.33
Rata-rata Turnaround Time: 15.00

Hasil juga disimpan ke file hasil_sjf.txt
madha_27@DESKTOP-S58N8I3:~$
```

Hasil yang disimpan:

```
madha_27@DESKTOP-S58N8I3:~$ cat hasil_sjf.txt
=== HASIL PENJADWALAN SJF ===
PID   AT   BT   CT   TAT   WT
P1     0   10   10   10     0
P2     1    5   15   14     9
P3     2    8   23   21    13

Average Waiting Time = 7.33
Average Turnaround Time = 15.00
madha_27@DESKTOP-S58N8I3:~$
```

4. Tugas 4 Eksperimen dengan data berbeda untuk perbandingan algoritma

a. Codingan

diambil dari praktikum yang round robin:

```

#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};
int main() {
    int n, quantum;
    printf("=== SIMULASI ALGORITMA ROUND ROBIN ===\n");
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);
    printf("Masukkan time quantum: ");
    scanf("%d", &quantum);
    struct Process proc[n];
    // Input data proses
    printf("\n");
    for(int i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        printf("Proses P%d\n", proc[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &proc[i].arrival_time);
        printf("Burst Time: ");
        scanf("%d", &proc[i].burst_time);
        proc[i].remaining_time = proc[i].burst_time;
        printf("\n");
    }
    int current_time = 0;
    int completed = 0;
    int queue[100], front = 0, rear = 0;
    bool in_queue[n];

```

b. Penjelasan algoritma

- Test case 1 proses dengan burst time bervariasi
Pada case ini tiap proses punya waktu eksekusi yang sangat berbeda jadi algoritma yang paling efisien adalah SJF karena proses dengan waktu paling singkat dikerjakan lebih dulu.
- Test case 2 semua proses datang bersamaan
Pada case ini menggunakan round robin karena lebih adil semua bagian giliran, tapi ada sedikit tambahan waktu karena switching.
- Test case 3 proses datang dengan interval waktu berbeda
Pada case ini proses datang tidak bersamaan jadi penggunaan algoritma SJF lebih efisien tapi bisa terjadi starvation kalau proses panjang terus tertunda oleh proses baru yang lebih pendek.

c. Hasil output program

- Output test case 1 proses dengan burst time bervariasi


```

=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 3

Proses P1
Arrival Time: 0
Burst Time: 24

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 3

Proses P4
Arrival Time: 3
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 21
Waktu 3: Menjalankan P1 -> Sisa waktu: 18
Waktu 6: Menjalankan P2 -> Selesai pada waktu 9
Waktu 9: Menjalankan P3 -> Selesai pada waktu 12
Waktu 12: Menjalankan P4 -> Sisa waktu: 3
Waktu 15: Menjalankan P1 -> Sisa waktu: 15
Waktu 18: Menjalankan P1 -> Sisa waktu: 12
Waktu 21: Menjalankan P1 -> Sisa waktu: 9
Waktu 24: Menjalankan P4 -> Selesai pada waktu 27
Waktu 27: Menjalankan P4 -> Selesai pada waktu 27

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT          | TAT | WT |
|-----|----|----|-------------|-----|----|
| P1  | 0  | 24 | -2109600109 | 0   | 0  |
| P2  | 1  | 3  | 9           | 8   | 5  |
| P3  | 2  | 3  | 12          | 10  | 7  |
| P4  | 3  | 6  | 27          | 24  | 18 |


Average Turnaround Time: 10.50
Average Waiting Time: 7.50
madha_27@DESKTOP-S58N8I3:~$ |

```

- Output test case2 proses datang bersamaan

```

madha_27@DESKTOP-S58N8I3:~$ gcc round_robin.c -o round_robin
madha_27@DESKTOP-S58N8I3:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 4

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 0
Burst Time: 5

Proses P3
Arrival Time: 0
Burst Time: 8

Proses P4
Arrival Time: 0
Burst Time: 12

Proses P5
Arrival Time: 0
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 4: Menjalankan P2 -> Sisa waktu: 1
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P4 -> Sisa waktu: 8
Waktu 16: Menjalankan P5 -> Sisa waktu: 2
Waktu 20: Menjalankan P1 -> Sisa waktu: 2
Waktu 24: Menjalankan P1 -> Selesai pada waktu 26
Waktu 26: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P2 -> Selesai pada waktu 27
Waktu 27: Menjalankan P3 -> Selesai pada waktu 31
Waktu 31: Menjalankan P3 -> Selesai pada waktu 31

```

```

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    26     26    16
P2      0      5    27     27    22
P3      0      8    31     31    23
P4      0     12   32766  424924400  32766
P5      0      6   716501594  0      0

Average Turnaround Time: 84984904.00
Average Waiting Time: 6565.40
madha_27@DESKTOP-S58N8I3:~$ |

```

- Output test case3 proses datang dengan interval

```

madha_27@DESKTOP-S58N8I3:~$ gcc round_robin.c -o round_robin
madha_27@DESKTOP-S58N8I3:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 5
Masukkan time quantum: 2

Proses P1
Arrival Time: 0
Burst Time: 8

Proses P2
Arrival Time: 2
Burst Time: 4

Proses P3
Arrival Time: 4
Burst Time: 9

Proses P4
Arrival Time: 6
Burst Time: 5

Proses P5
Arrival Time: 8
Burst Time: 3

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 2: Menjalankan P1 -> Sisa waktu: 4
Waktu 4: Menjalankan P2 -> Sisa waktu: 2
Waktu 6: Menjalankan P1 -> Sisa waktu: 2
Waktu 8: Menjalankan P1 -> Selesai pada waktu 10
Waktu 10: Menjalankan P3 -> Sisa waktu: 7
Waktu 12: Menjalankan P1 -> Selesai pada waktu 12
Waktu 12: Menjalankan P2 -> Selesai pada waktu 14
Waktu 14: Menjalankan P4 -> Sisa waktu: 3
Waktu 16: Menjalankan P2 -> Selesai pada waktu 16
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      8    16     16     8
P2      2      4    16     14    10
P3      4      9  -399642616  29486  727721632
P4      6      5    32765  -536419952  32765
P5      8      3  -399646118     0      0

Average Turnaround Time: -107278096.00
Average Waiting Time: 145550880.00
madha_27@DESKTOP-S58N8I3:~$ |

```

5. Tugas Analisis eksperimen perbandingan algoritma

a. Analisis performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
FCFS memberikan hasil terbaik ketika semua proses datang hampir bersamaan dan memiliki burst time yang mirip.
- Mengapa SJF hampir selalu memberikan Average WT terkecil?

Karena SJF selalu memilih proses dengan waktu eksekusi paling singkat terlebih dahulu. Proses kecil dapat selesai lebih cepat dan tidak menunggu proses Panjang.

- Apa trade-off dari round robin?
Round robin memberikan keadilan bagi semua proses karena setiap proses mendapat giliran eksekusi dalam jatah waktu tertentu.
Namun, trade-off nya Adalah meningkatnya jumlah context switch.

b. Context switching

- Algoritma mana yang paling banyak melakukan context switch?
Algoritma round robin paling banyak melakukan context switch karena cpu berganti proses setiap kali quantum habis, walau proses belum selesai.
- Bagaimana pengaruh quantum terhadap jumlah context switch di round robin?
Semakin kecil nilai quantum, semakin sering context switch terjadi, karena proses cepat sekali diganti sebelum sempat menyelesaikan tugasnya. Sebaliknya, semakin besar quantum jumlah context switch semakin sedikit, tetapi responsivitas menurun.
- Coba jalankan round robin dengan quantum 2,4,8,16 apa yang terjadi?
Untuk quantum 2:

```

madha_27@DESKTOP-S58N8I3:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 2

Proses P1
Arrival Time: 0
Burst Time: 2

Proses P2
Arrival Time: 0
Burst Time: 7

Proses P3
Arrival Time: 0
Burst Time: 9

Proses P4
Arrival Time: 0
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Selesai pada waktu 2
Waktu 2: Menjalankan P2 -> Sisa waktu: 5
Waktu 4: Menjalankan P3 -> Sisa waktu: 7
Waktu 6: Menjalankan P4 -> Sisa waktu: 4
Waktu 8: Menjalankan P2 -> Sisa waktu: 3
Waktu 10: Menjalankan P2 -> Sisa waktu: 1
Waktu 12: Menjalankan P3 -> Sisa waktu: 5
Waktu 14: Menjalankan P3 -> Sisa waktu: 3
Waktu 16: Menjalankan P4 -> Sisa waktu: 2
Waktu 18: Menjalankan P4 -> Selesai pada waktu 20
Waktu 20: Menjalankan P2 -> Selesai pada waktu 21
Waktu 21: Menjalankan P2 -> Selesai pada waktu 21

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      2      2      2      0
P2      0      7     21     21     14
P3      0      9    1379274976  32766  -1968804976
P4      0      6     20     20     14

Average Turnaround Time: 8202.25
Average Waiting Time: -492201248.00
madha_27@DESKTOP-S58N8I3:~$ |

```

Quantum 4:

```

madha_27@DESKTOP-S58N8I3:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 4

Proses P1
Arrival Time: 0
Burst Time: 4

Proses P2
Arrival Time: 0
Burst Time: 4

Proses P3
Arrival Time: 0
Burst Time: 4

Proses P4
Arrival Time: 0
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Selesai pada waktu 4
Waktu 4: Menjalankan P2 -> Selesai pada waktu 8
Waktu 8: Menjalankan P3 -> Selesai pada waktu 12
Waktu 12: Menjalankan P4 -> Sisa waktu: 2
Waktu 16: Menjalankan P4 -> Selesai pada waktu 18

=== HASIL PENJADWALAN ROUND ROBIN ===

```

PID	AT	BT	CT	TAT	WT
P1	0	4	4	4	0
P2	0	4	8	8	4
P3	0	4	12	12	8
P4	0	6	18	18	12

```

Average Turnaround Time: 10.50
Average Waiting Time: 6.00
madha_27@DESKTOP-S58N8I3:~$ |

```

Quantum 8:

```

Average Waiting Time: 0.00
madha_27@DESKTOP-S58N8I3:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 8

Proses P1
Arrival Time: 2
Burst Time: 7

Proses P2
Arrival Time: 0
Burst Time: 9

Proses P3
Arrival Time: 2
Burst Time: 0

Proses P4
Arrival Time: 0
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P2 -> Sisa waktu: 1
Waktu 8: Menjalankan P4 -> Selesai pada waktu 14
Waktu 14: Menjalankan P1 -> Selesai pada waktu 21
Waktu 21: Menjalankan P2 -> Selesai pada waktu 22
Waktu 22: Menjalankan P2 -> Selesai pada waktu 22

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      2      7    21     19     12
P2      0      9    22     22     13
P3      2      0   1045528800  32764   -1115402048
P4      0      6    14     14      8

Average Turnaround Time: 8204.75
Average Waiting Time: -278850496.00
madha_27@DESKTOP-S58N8I3:~$ |

```

Quantum 16:

```

madha_27@DESKTOP-S58N8I3:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 16

Proses P1
Arrival Time: 0
Burst Time: 4

Proses P2
Arrival Time: 0
Burst Time: 4

Proses P3
Arrival Time: 2
Burst Time: 0

Proses P4
Arrival Time: 0
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Selesai pada waktu 4
Waktu 4: Menjalankan P2 -> Selesai pada waktu 8
Waktu 8: Menjalankan P4 -> Selesai pada waktu 14
|

```

Untuk quantum 16 stuck di seperti Digambar.

c. Fairness

- Algoritma mana yang paling adil untuk semua proses?

Round robin paling adil karena setiap proses mendapat jatah waktu cpu yang sama tanpa memandang Panjang brush-nya.

- Proses mana yang paling dirugikan di algoritma FCFS?
Proses dengan brush time pendek yang datang setelah proses Panjang. Mereka harus menunggu proses besar selesai sepenuhnya sebelum bisa dijalankan.
- Apakah ada kemungkinan starvation d SJF?
Ya, ada. Starvation bisa terjadi Ketika banyak proses kecil terus datang, menyebabkan proses besar tidak pernah dapat giliran.

d. Real world application

- Untuk web srver yang melayani banyak request kecil, algoritma mana yang cocok?
SJF cocok, karena Sebagian besar permintaan web cepat diproses, dan prioritas diberikan pada reques kecil agar respons lebih cepat.
- Untuk render video(task besar), algoritma mana yang cocok?
FCFS lebih cocok karena tugas besar seperti rendering video tidak butuh interaksi pengguna dan dapat berjalan penuh sampai selesai tanpa perlu sering berpindah konteks.
- Untuk OS desktop(interactive), algoritma manna yang cocok?
Round robin paling cocok untuk system interaktif, karena bisa menangani banyak proses pengguna secara adil , dengan quantum yang seimbang system terasa responsive tanpa proses tertentu mendominasi cpu.

1.4.KESIMPULAN

Setiap algoritma punya kelebihan dan kekurangan tergantung situasi.

SJF (Shortest Job First) paling cepat menyelesaikan proses rata-rata karena selalu mendahulukan pekerjaan yang singkat, tapi bisa membuat proses panjang menunggu lama.

FCFS (First Come First Serve) mudah dipahami dan dijalankan, tapi bisa tidak efisien kalau proses pertama terlalu lama.

Round Robin paling adil karena semua proses dapat giliran, cocok untuk sistem multitasking seperti komputer sehari-hari, meski waktu totalnya sedikit lebih lama karena sering berganti proses.

Secara keseluruhan, tidak ada algoritma yang paling sempurna, semuanya tergantung pada kebutuhan sistem dan kondisi proses yang dijalankan.